LAPORAN TUGAS PERKULIAHAN
PENGEMBANGAN SISTEM BERBASIS WEB_14823293

Kelas : R

Gara Zaki Pramudya 1482500069

Dosen Pengampu

Abdullah Faqih Septiyanto, S.Kom., M.Kom.

PROGRAM STUDI SISTEM TEKNOLOGI & INFORMASI

NOVEMBER 2026

DAFTAR ISI

BAB I PENDAHULUAN 3
1.1 Tujuan Tugas 3
1.2 Studi Kasus 3
BAB II STRUKTUR PROJECT & ROUTING 4
2.1 Struktur Direktori 4
2.2 Daftar Endpoint (Routing) 4
BAB III PEMBAHASAN KODE & IMPLEMENTASI 5
3.1 Persiapan Data 5
3.2 Implementasi Routing 5
3.3 Implementasi Jinja2 Template Inheritance 5
3.4 Perulangan (For Loop) & Filter di Jinja2 6
3.5 Percabangan (If Condition) pada Validasi Username 6
BAB IV HASIL PENGUJIAN 7
4.1 Halaman Beranda 7
4.2 Halaman Daftar Proyek 7
4.3 Halaman Detail Proyek 8
4.4 Fitur Cek Username 8
4.5 Custom Error 404 9
BAB V KESIMPULAN 10
LAMPIRAN Kode Program app.py 11

BAB I
PENDAHULUAN

1.1 Tujuan Tugas

Tujuan dari project ini adalah mengembangkan aplikasi berbasis web sederhana menggunakan framework Flask dengan mengimplementasikan dua materi utama, yaitu:

1. **Routing (Pertemuan 9):** menerapkan route statis, dynamic route dengan converter tipe data, URL building (url_for), redirect, penanganan HTTP method, serta custom halaman error 404.
2. **Templating (Pertemuan 10):** menggunakan Jinja2 Template Engine untuk memisahkan logika Python dengan tampilan HTML, menerapkan Template Inheritance (base.html), serta menggunakan Control Structure (if dan for) beserta Jinja2 filter.

1.2 Studi Kasus

Studi kasus yang digunakan adalah Web Portofolio Pribadi. Aplikasi ini memuat profil diri, daftar proyek/tugas kuliah, serta simulasi fitur pengecekan ketersediaan username (client/system profile) dengan mengambil sampel data dummy seperti "gara", "indira", "zigma", dan "arga".

BAB II
STRUKTUR PROJECT & ROUTING

2.1 Struktur Direktori

Aplikasi menggunakan pemisahan antara file logika (app.py) dan tampilan (folder templates/) dengan susunan sebagai berikut:

- `app.py` : file utama aplikasi Flask.
- `templates/base.html` : template induk (navbar dan footer).
- `templates/beranda.html, tentang.html, proyek.html, kontak.html` : halaman statis.
- `templates/detail_proyek.html, cek.html` : halaman dinamis.
- `templates/404.html` : halaman custom error.

2.2 Daftar Endpoint (Routing)

Aplikasi memiliki endpoint berikut yang dikonfigurasi melalui dekorator @app.route. Seluruh endpoint hanya menerima method GET (default Flask).

No	Route	Method	Fungsi / Keterangan
1	/ dan /beranda	GET	Menampilkan halaman utama dan form input username.
2	/tentang	GET	Menampilkan profil dan list keahlian.
3	/proyek	GET	Menampilkan tabel daftar proyek.
4	/proyek/<int:id>	GET	Dynamic route detail proyek berdasarkan ID; abort(404) jika ID tidak valid.
5	/kontak	GET	Menampilkan kontak email.
6	/cek	GET	Menangkap query string (?u=) dari form lalu melakukan redirect.
7	/cek/<username>	GET	Dynamic route dengan validasi panjang karakter dan ketersediaan nama dari list TERDAFTAR.

BAB III
PEMBAHASAN KODE & IMPLEMENTASI

3.1 Persiapan Data

Sumber data pada aplikasi berupa data dummy yang disimpan dalam variabel Python. Variabel TERDAFTAR bertipe list berisi username yang sudah terpakai ("gara", "indira", "zigma", "arga"), sedangkan variabel PROYEK bertipe dictionary dengan key berupa ID proyek dan value berupa dictionary berisi judul, kategori, status, dan tahun proyek.

3.2 Implementasi Routing

Route statis dibuat dengan dekorator `@app.route()`. Fungsi beranda() memiliki dua route sekaligus, yaitu `/` dan `/beranda`, dan mengirim variabel nama dan peran ke template melalui render_template().

Dynamic route digunakan pada `/proyek/<int:id>`. Converter `int` memastikan ID bertipe integer, sehingga URL dengan nilai non-angka otomatis menghasilkan 404. Data proyek diambil dengan `PROYEK.get(id)`; jika hasilnya None maka dipanggil `abort(404)`.

Fitur cek username memakai dua route. Route `/cek` menangkap query string `request.args.get("u")` dari form; bila kosong, pengguna di-redirect ke beranda, bila terisi diarahkan ke `/cek/<username>` menggunakan `redirect(url_for(...))`. Route kedua mengubah teks menjadi huruf kecil, lalu menghitung dua variabel logika:

- `valid = len(u) >= 4` : username minimal 4 karakter.
- `tersedia = valid and (u not in TERDAFTAR)` : valid dan belum ada di list TERDAFTAR.

Halaman error 404 ditangani oleh custom error handler `@app.errorhandler(404)` yang menampilkan template 404.html dengan status code 404.

3.3 Implementasi Jinja2 Template Inheritance

Aplikasi menggunakan file base.html sebagai parent template yang memuat kerangka HTML, Bootstrap, navigasi dinamis dengan `{{ url_for() }}`, dan

blok `{% block content %}`. Semua halaman child memanggilnya menggunakan `{% extends "base.html" %}` sehingga desain web konsisten dan tidak ada pengulangan kode (prinsip DRY).

3.4 Perulangan (For Loop) & Filter di Jinja2

Pada halaman Daftar Proyek (proyek.html), data bertipe dictionary yang dikirim dari app.py diiterasi menggunakan Jinja2:

- `{% for id, p in data_proyek.items() %}` : membongkar dictionary; variabel id menyimpan key proyek dan p menyimpan value (detail proyek).
- `{{ p.judul | upper }}` : filter upper mengubah teks judul menjadi huruf kapital saat dirender.
- `{{ loop.index }}` : mencetak nomor urut otomatis.

3.5 Percabangan (If Condition) pada Validasi Username

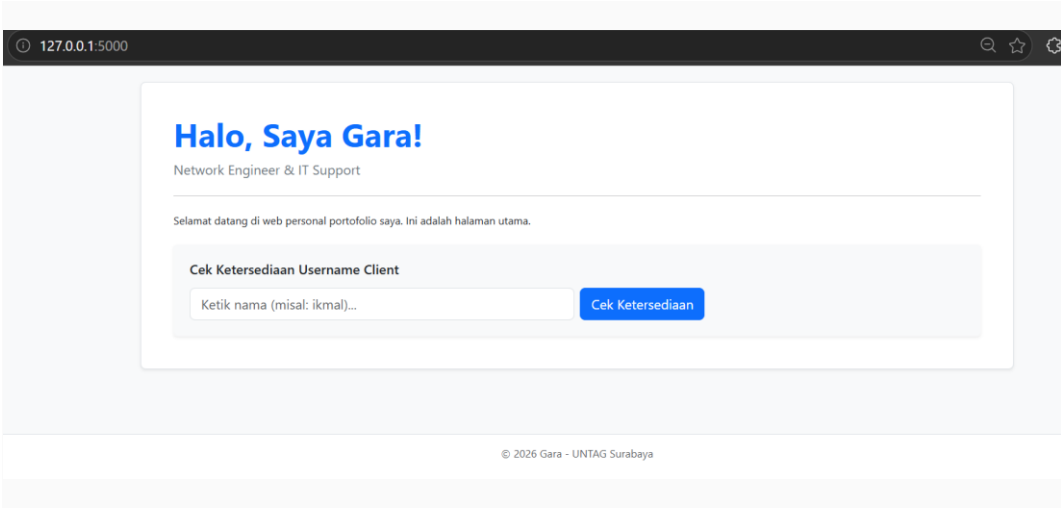
Pada halaman cek.html terdapat logika percabangan berdasarkan variabel valid (minimal 4 karakter) dan tersedia (tidak ada di list TERDAFTAR):

- `{% if not valid %}` : menampilkan peringatan bahwa username tidak valid.
- `{% elif tersedia %}` : menampilkan konfirmasi bahwa username tersedia.
- `{% else %}` : menampilkan peringatan bahwa nama sudah terpakai.

BAB IV
HASIL PENGUJIAN

Pengujian dilakukan dengan menjalankan aplikasi pada server pengembangan Flask (debug mode) dan mengakses setiap halaman melalui browser. Hasil pengujian ditunjukkan sebagai berikut.

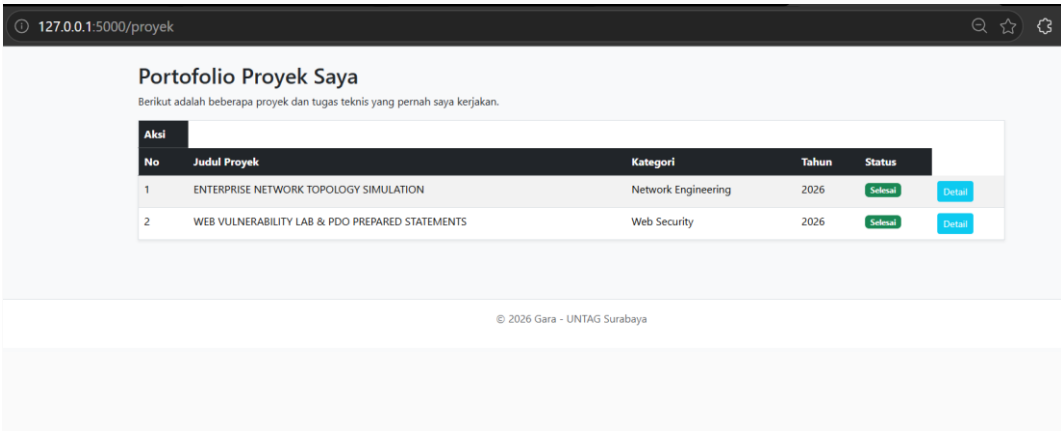
4.1 Halaman Beranda



Gambar 1. Tampilan Halaman Beranda

4.2 Halaman Daftar Proyek

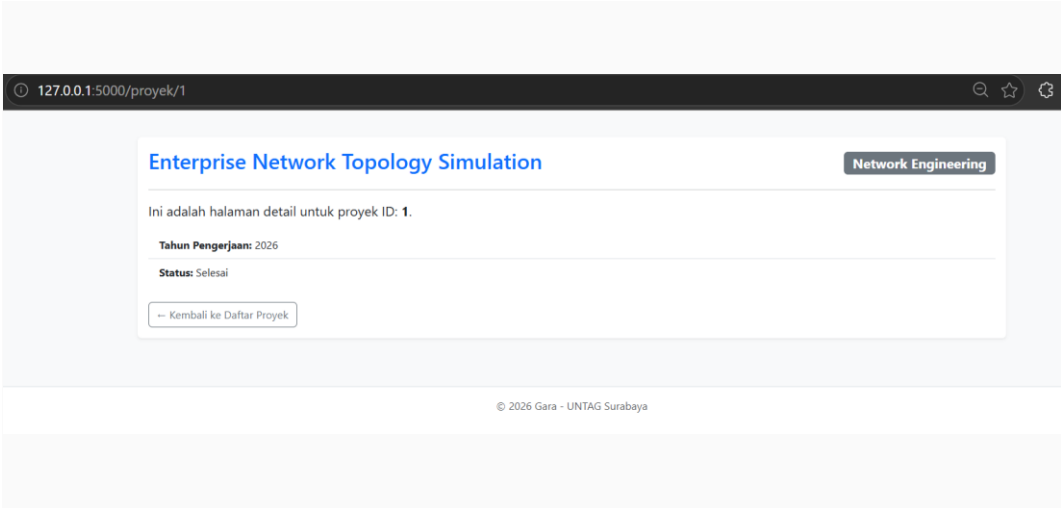
Halaman ini menunjukkan hasil perulangan for dan filter upper (judul tampil dalam huruf kapital).



Gambar 2. Tampilan Halaman Daftar Proyek

4.3 Halaman Detail Proyek

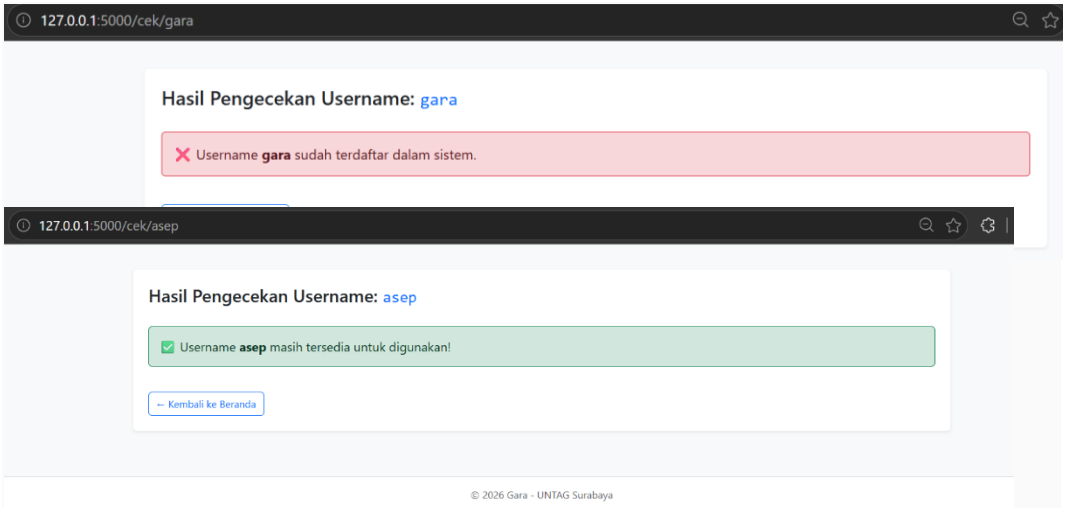
Pengujian dynamic route dengan converter int pada URL /proyek/1.



Gambar 3. Tampilan Halaman Detail Proyek

4.4 Fitur Cek Username

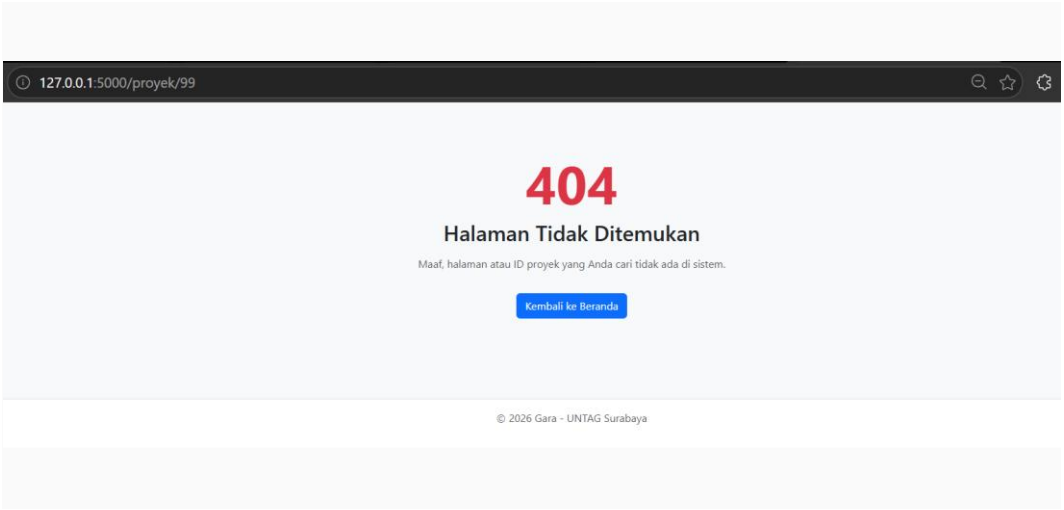
Pengujian fitur cek username melalui form dan route dinamis /cek/<username>.



Gambar 4. Pengujian Fitur Cek Username

4.5 Custom Error 404

Pengujian halaman error kustom dengan mengakses ID proyek yang tidak ada.



Gambar 5. Tampilan Custom Error 404

BAB V
KESIMPULAN

Implementasi Routing di Flask mempermudah pengelolaan URL secara dinamis dan aman, terutama dengan dukungan fungsi url_for dan proteksi tipe data melalui converter. Dipadukan dengan Jinja2 Templating, pengembangan web menjadi lebih efisien karena struktur HTML cukup ditulis sekali dan injeksi data dari logika Python dapat dilakukan secara aman langsung ke sisi frontend.

LINK GIT HUB : https://github.com/GaraNEIW/Portofolio-Flask-Gara-_-Tugas-Individu-5/tree/main/templates

LAMPIRAN
Kode Program app.py

app.py | cek.html | Beranda - Gara | base.html | beranda.html | 404.html

app.py > ...

```python
# 1. IMPORT MODUL
from flask import Flask, render_template, request, redirect, url_for, abort

# 2. INISIALISASI APLIKASI
app = Flask(__name__)

# 3. PERSIAPAN DATA (Sumber Data Dummy)
# Variabel: TERDAFTAR (Tipe data: List)
TERDAFTAR = ["gara", "indira", "zigma", "arga"]

# Variabel: PROYEK (Tipe data: Dictionary / Kamus)
PROYEK = {
    1: {
        "judul": "Enterprise Network Topology Simulation",
        "kategori": "Network Engineering",
        "status": "Selesai",
        "tahun": 2026
    },
    2: {
        "judul": "Web Vulnerability Lab & PDO Prepared Statements",
        "kategori": "Web Security",
        "status": "Selesai",
        "tahun": 2026
    }
}

# 4. ROUTE STATIS PERTAMA (Halaman Beranda)
@app.route("/")
@app.route("/beranda")
def beranda():
    return render_template("beranda.html", nama="Gara", peran="Network Engineer & IT Support")

@app.route("/proyek")
def daftar_proyek():
    return render_template("proyek.html", data_proyek=PROYEK)

@app.route("/proyek/<int:id>")
def detail_proyek(id):
    # Mengambil data proyek spesifik berdasarkan ID
    item = PROYEK.get(id)

    # Validasi: Jika data (ID) tidak ditemukan di dictionary
    if item is None:
        abort(404)  # Memunculkan error 404 Not Found

    return render_template("detail_proyek.html", item=item, id=id)

    # ========================================
# HALAMAN TENTANG & KONTAK
# ========================================
@app.route("/tentang")
def tentang():
    # Mengirim list keahlian ke HTML
    list_keahlian = ["Network Engineering", "Web Development", "System Troubleshooting"]
    return render_template("tentang.html", keahlian=list_keahlian)

@app.route("/kontak")
def kontak():
    # Mengirim data string email ke HTML
    return render_template("kontak.html", email="gara@sby.centrin.net.id")


# ========================================
# FITUR CEK USERNAME CLIENT / SYSTEM PROFILE
# ========================================

# 1. Menangkap data dari form pencarian (Query String)
@app.route("/cek")
def cek_query():
    # Mengambil nilai dari input form yang bernama 'u'
    u = request.args.get("u", "")

    # Jika input kosong, kembalikan ke beranda
    if not u:
        return redirect(url_for("beranda"))

    # Jika ada isinya, alihkan ke route dinamis di bawah ini
    return redirect(url_for("cek_username", username=u))


# 2. Route Dinamis untuk memproses logika ketersediaan
@app.route("/cek/<username>")
```

```python
# 2. Route Dinamis untuk memproses logika ketersediaan
@app.route("/cek/<username>")
def cek_username(username):
    # Mengubah teks menjadi huruf kecil semua agar seragam
    u = username.lower()

    # Logika 1: Username harus minimal 4 karakter (menghasilkan True/False)
    valid = len(u) >= 4

    # Logika 2: Cek apakah valid DAN tidak ada di dalam list TERDAFTAR
    tersedia = valid and (u not in TERDAFTAR)

    # Melempar 3 variabel ke HTML: username, valid, dan tersedia
    return render_template("cek.html", username=u, valid=valid, tersedia=tersedia)

# ========================================
# CUSTOM ERROR HANDLER (Pertemuan 9)
# ========================================
@app.errorhandler(404)
def halaman_tidak_ditemukan(e):
    return render_template("404.html"), 404

# 5. MENJALANKAN SERVER FLASK
if __name__ == "__main__":
    app.run(debug=True)
```